MiniSat TM/TC

Building a CubeSat's command & telemetry system from scratch: flight software in C, a ground station in Python, and a simulated radio link

Mohammed Yusuf
B.Tech Electronics & Communication Engineering, NIT Puducherry

Personal project, October 2026

2,170
lines of C and Python

34 / 34
tests passing

6.4 h
simulated mission day, 5 ground passes

35
commands, every one verified

97%
commands delivered at 1 bit error in 1000

0
missed 1–3-bit errors in 150,000 trials

Inspired by my internship on a CubeSat On-Board Computer team. Contains no company or mission data: every number and format here is my own, built on public standards.

1. What I built, in plain words

A satellite can't be touched after launch. The only way to control it is to send it **commands** (telecommands, TC) over radio, and the only way to know how it's doing is to read the **telemetry** (TM) it sends back. This project builds both ends of that conversation and the radio link in between:

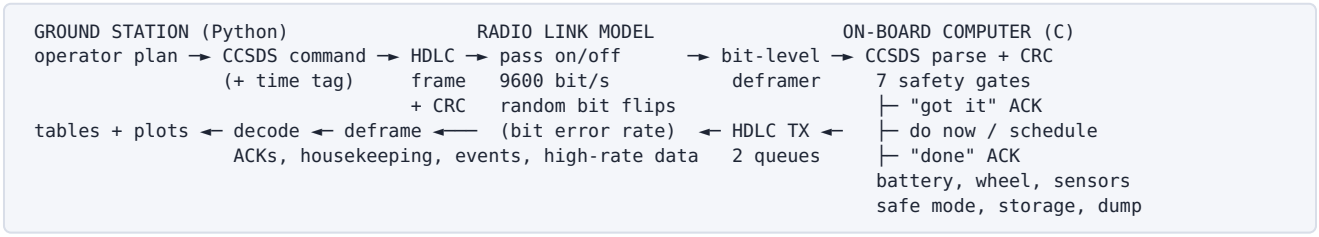
Part	Language	What it does
Flight software (the satellite's brain)	C99	Receives radio bits, finds frames, checks them, runs or schedules commands, reports back, looks after the battery, records data while nobody is listening.
Ground station	Python	Builds and sends commands, retries when replies go missing, decodes every kind of telemetry into tables.
Link + mission simulator	Python	Decides when the satellite is overhead (ground passes), limits speed to 9600 bit/s, and flips random bits to imitate a noisy radio.

> **Think of it like…**
> A hotel reception desk (the satellite's computer) that only gets phone calls for 10 minutes every 90 minutes. It has to check every request is genuine, pass it to the right department, confirm twice ("got it", "done"), keep a diary of everything that happens while the phone is silent, and read that diary out at the next call.

Why I built it

During my internship I documented this kind of system: packet formats, command flowcharts and verification. I wanted to prove I understood it well enough to build the whole thing myself, end to end, and to measure how well it works rather than just describe it.

2. The big picture



How the two programs stay in step: every 100 ms of simulated time, the ground hands the satellite the uplink bits for that slice. The satellite runs ten 10 ms "ticks" (physics, scheduler, telemetry, transmitter) and hands back the downlink bits it produced. This is called **software-in-the-loop** testing: the real flight code runs, only the hardware around it is simulated. A 6.4-hour mission runs in about 2 seconds.

The message "envelopes"

Layer	Job	Where in the code
HDLC / AX.25 frame	Marks where a message starts and ends on the radio (flag 0x7E), carries addresses, has its own CRC (FCS).	`hdlc.c`, `link.py`
CCSDS space packet	Says which application it's for (APID), counts packets (sequence count), carries a time, has a second CRC.	`ccsds.c`, `link.py`
Application data	Command ID + parameters, or telemetry values.	`obc.c`, `station.py`

Full byte-by-byte formats are in `docs/ICD.md`.

3. The building blocks

3.1 The ICD as code: one source of truth

Every number both sides must agree on (APIDs, command IDs, error codes, sizes) lives in one C header, `icd.h`. The Python ground station **reads that same file** when it starts. Change a number once and both sides change.

> **Lesson from my internship**
> Most real integration bugs are "two sides disagree": the code says one APID, the spreadsheet another. Making the ICD executable removes that whole class of bug.

3.2 CRC-16: spotting damaged messages

A CRC is a 16-bit fingerprint calculated from the message. The receiver recalculates it; if it doesn't match, the message was damaged. I use CRC-16/CCITT-FALSE (polynomial 0x1021, start 0xFFFF). The standard test: the text "123456789" must give `0x29B1`. Both the C and Python versions pass it.

3.3 Big-endian, without buffer overflows

Numbers are sent "big end first" (4500 = `11 94`). Instead of raw pointer arithmetic, all reading and writing goes through a small **cursor** that knows the buffer size and sets an error flag instead of running past the end. A whole family of bugs (overruns, reading garbage) can't happen.

3.4 Bit stuffing: the trick that makes framing work

The frame marker 0x7E is `01111110`: six 1s in a row. If data ever had six 1s, the receiver would think the frame ended. So the sender adds a 0 after every five 1s, and the receiver removes it. My receiver works **one bit at a time** like real radio hardware: it counts 1s, drops stuffed 0s, spots flags, and treats seven 1s as "line idle, start again". The unit test feeds it the worst possible data (all 0x7E and 0xFF bytes) and checks it comes out perfect.

3.5 CCSDS packets and time

Each packet starts with a 6-byte header: type (command or telemetry), APID (the "department"), a 14-bit sequence count, and the length (stored as length minus one, a classic trap). Telemetry carries the satellite's clock in **CUC format** (seconds + fractions). Commands can carry a **time tag** saying when to run.

4. The flight software: the life of a command

Every command passes through seven gates before anything moves. Fail any gate and it stops there, with a reason sent back to the ground.

#	Gate	If it fails
1	Frame and packet CRC correct	Thrown away silently: a damaged message can't be trusted, not even its address. Ground notices the missing ACK and retries.
2	APID is in the routing table	Reject: BAD_APID
3	Not a repeat of the last sequence count	Reject: DUPLICATE (protects against replays and double execution)
4	Known command, right parameter length	Reject: BAD_CMD / BAD_LENGTH
5	Allowed in the current mode	Reject: MODE (e.g. no wheel commands in SAFE)
6	Every parameter in range	Reject: BAD_PARAM (e.g. wheel 9000 rpm, SSTM rate 0 Hz)
7	Time tag not in the past, schedule has room	Reject: TIME_PAST / SCHED_FULL

Then the satellite sends the **acceptance ACK** and either runs the command now, or stores it in a 16-slot schedule. When it finishes it sends the **execution ACK** (or an execution failure, like a TLE whose checksum is wrong).

> **Why two ACKs?**
> Like food delivery: "order received" isn't "order delivered". If you only get the first, you know exactly where things went wrong.

Features that make it behave like a real satellite

- **Time-tagged commands** run when their time arrives (using "now ≥ time", so a slow tick can never skip one), and the satellite **re-checks the mode at that moment**. Their execution ACKs are also written to on-board storage, because nobody is usually listening when they run.
- **FDIR safe mode**: if battery voltage stays under the threshold for 3 seconds, the satellite switches off the payload, stops the wheel and enters SAFE. Leaving SAFE needs a command and a battery 300 mV above the threshold (hysteresis, so it doesn't flip back and forth).
- **Housekeeping** every 10 s, sent live and also recorded in a 600-slot store. A **DUMP** command plays it back, paced by the transmitter so it never floods the queue.
- **SSTM** (super-sampled telemetry): a short burst of 1–50 Hz recording, a small state machine (idle → logging → idle + "done" event). Overlapping sessions are refused.
- **TLE upload** with the official checksum rule (sum the digits, minus signs count 1, mod 10) on both lines.
- **Transmitter** limited to 9600 bit/s with two queues: ACKs and events jump ahead of bulk data.
- **Simple physics**: orbit with 61% sunlight, solar panel, battery state of charge, loads that change with what's switched on, a reaction wheel that speeds up with realistic limits, a magnetometer that the magnetorquer disturbs, and temperature.

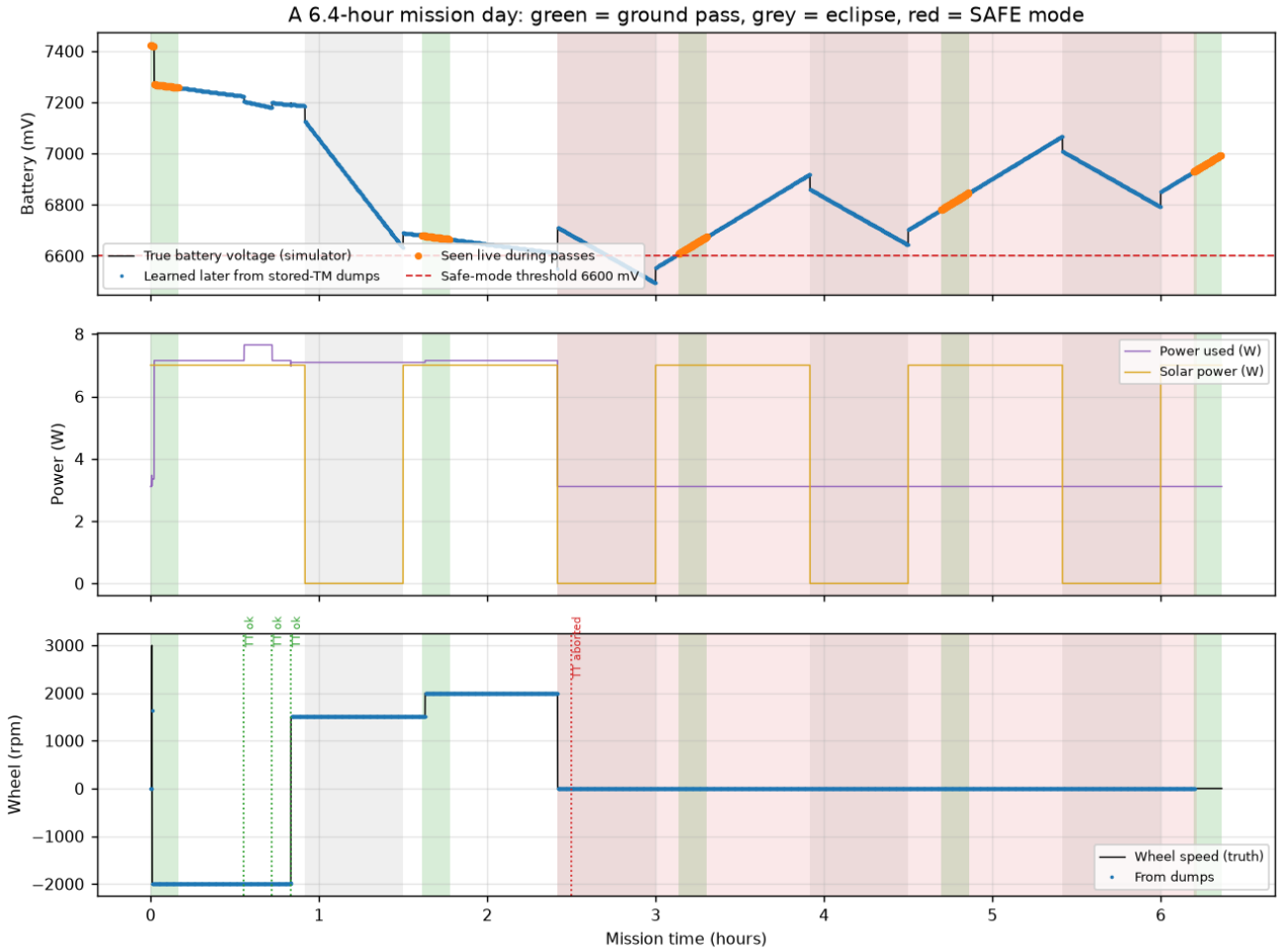
5. The ground station and the link

The ground station is a completely separate Python implementation of framing and packets. If two independently written programs understand each other perfectly, the format is unambiguous.

- **Passes**: the link only works during five 10-minute windows across the day. Outside them, everything is lost (as in reality).
- **Bit errors**: each bit on the air can be flipped with a chosen probability (BER), in both directions.
- **Automatic retries**: no acceptance ACK within the timeout → send the same packet again, up to 4 times. If the reply is DUPLICATE, the ground knows the first copy did arrive and only its ACK was lost, so it marks the command as accepted. The duplicate check on board makes retries safe.
- **Loss detection**: the ground counts missing telemetry from gaps in the sequence count, exactly what sequence counts are for.

6. Results

6.1 A mission day

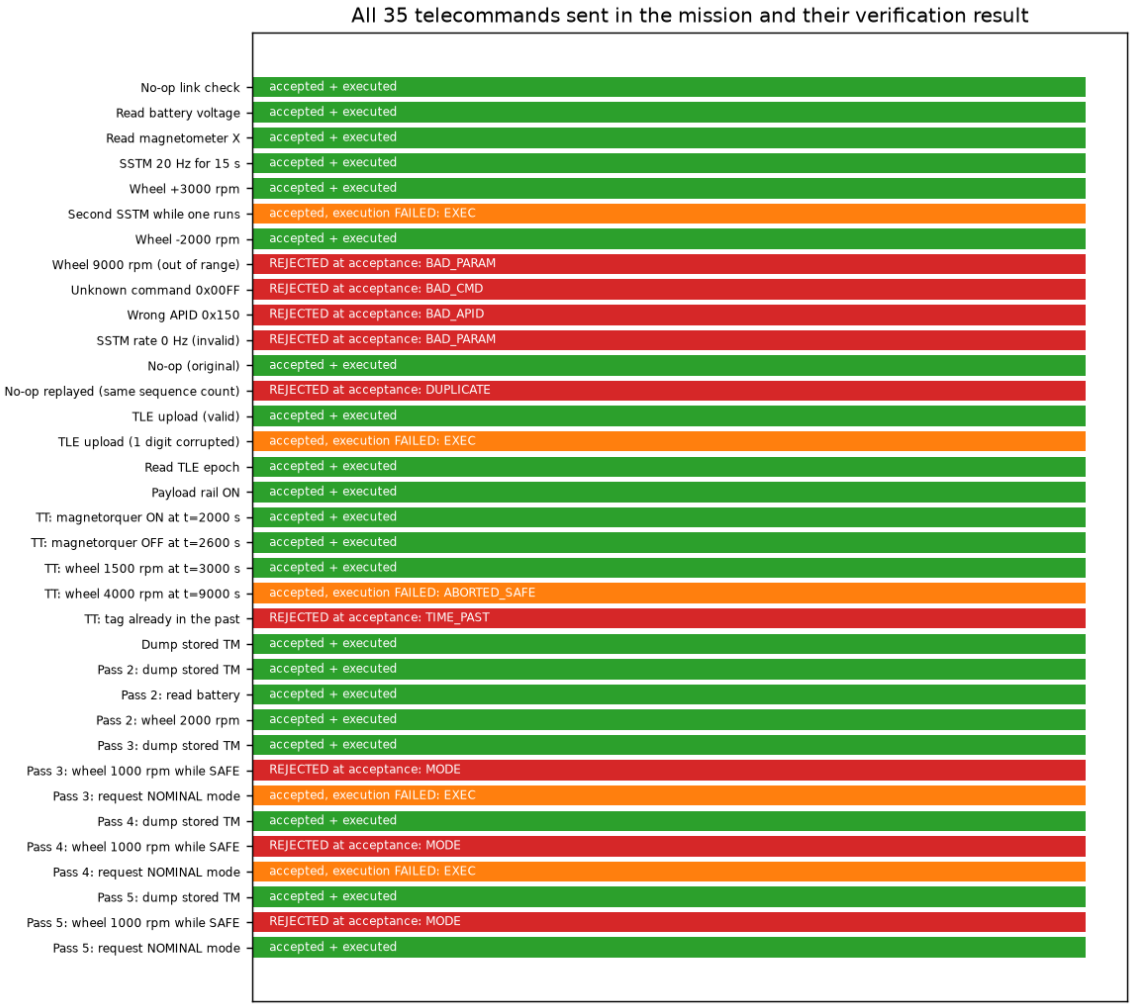


Black line = what really happened inside the simulator. Orange = what the ground saw live. Blue = what the ground learned later from stored-telemetry dumps.

The story: in pass 1 the operator checks out the satellite, tests the wheel, uploads orbit data, switches the payload on and leaves four time-tagged commands. The payload draws more power than the panels give, so the battery falls each orbit. At 2.42 h (8,703 s), out of contact, the voltage crosses 6,600 mV for 3 s and the satellite **puts itself into SAFE mode** and sheds load. The time-tagged wheel command due at 9,000 s is **aborted** because of it. In passes 3 and 4 the operator asks for NOMINAL but the battery hasn't recovered enough, so the request is refused (hysteresis). In pass 5 it succeeds.

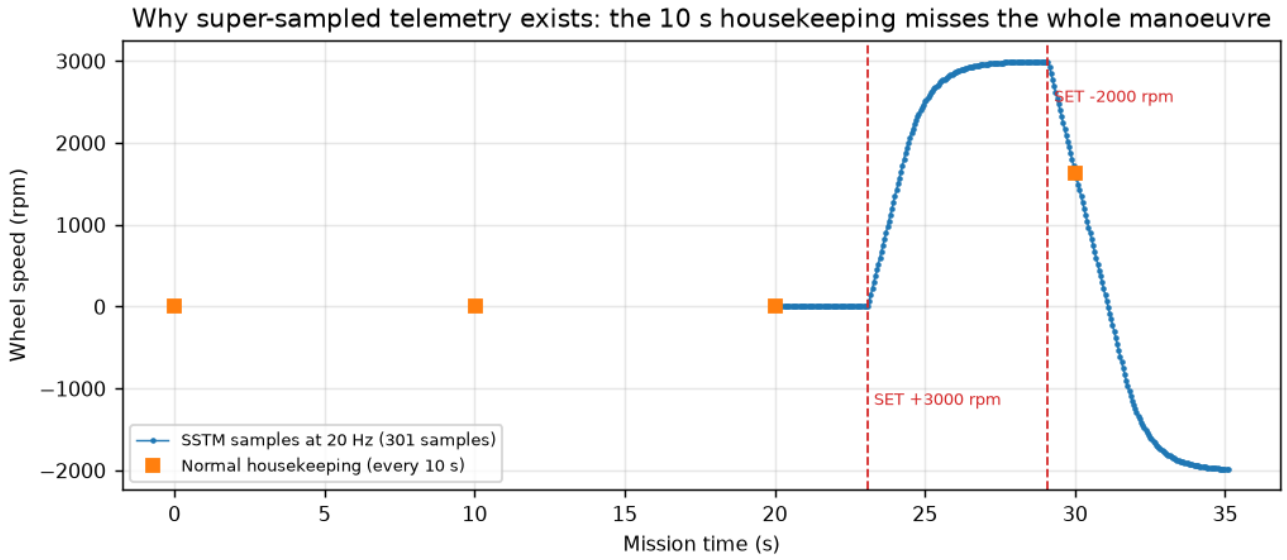
> **Only 13% of the timeline was seen live**
> The other 87% of the satellite's history reached the ground through on-board storage and dumps, including the safe-mode event itself. This is why stored telemetry exists.

6.2 Every command, verified



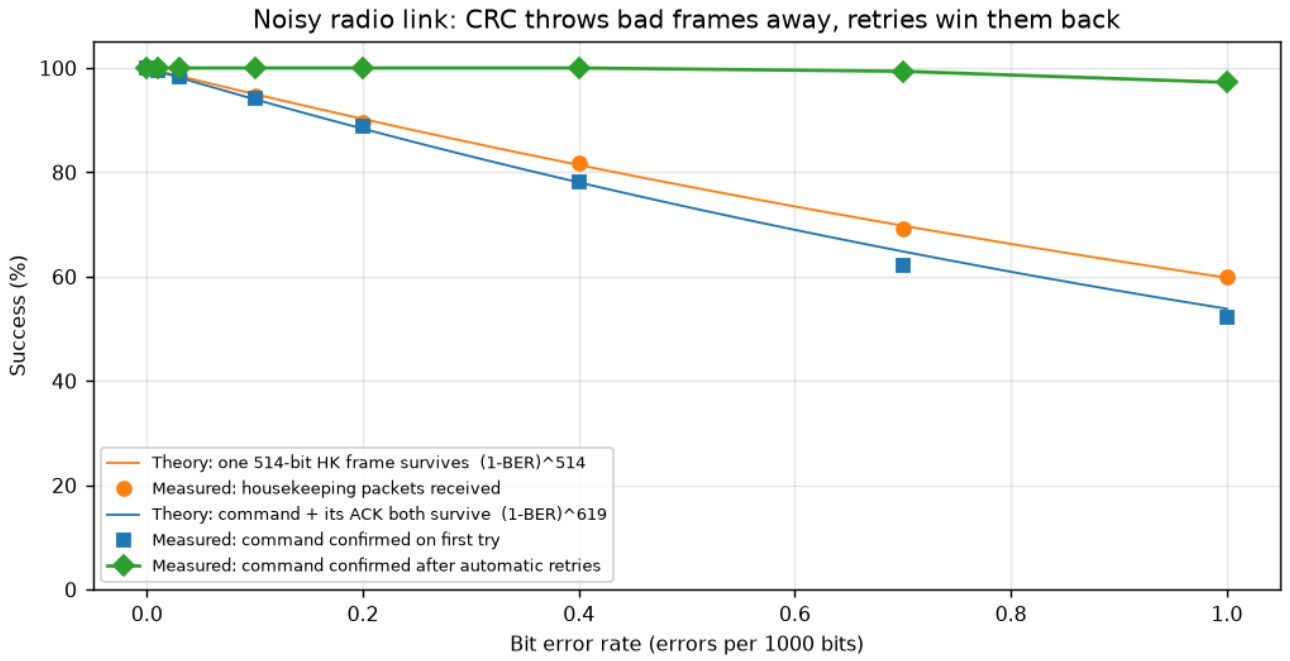
35 commands: 21 accepted and executed, 9 refused at acceptance (each with the right reason: BAD_PARAM, BAD_CMD, BAD_APID, DUPLICATE, TIME_PAST, MODE), 5 accepted but failed on execution for good reasons (corrupted TLE, overlapping SSTM, aborted in SAFE, battery too low to leave SAFE).

6.3 Why high-rate telemetry exists



The wheel is commanded to +3000 rpm, then −2000 rpm. Normal 10-second housekeeping (orange squares) catches almost nothing. The 20 Hz SSTM session (301 samples) shows the whole response: the speed ramps at its acceleration limit and settles smoothly.

6.4 A noisy radio



Each point: about 470 commands and 1,190 housekeeping packets over 20 simulated minutes.

Bit error rate	Commands OK first try	Theory	After retries	Housekeeping received	Theory
0	100%	100%	100%	100%	100%
1 in 10,000	94.3%	94.0%	100%	94.6%	95.0%
4 in 10,000	78.1%	78.1%	100%	81.8%	81.4%
1 in 1,000	52.1%	53.8%	97.2%	60.0%	59.8%

The theory is simple: a frame of N bits survives if every bit survives, so the chance is $(1 - \mathrm{BER})^N$. A housekeeping frame is 514 bits on air; a command plus its ACK is 619. The measurements match the theory closely, which shows the framing, CRC and decoding behave exactly as designed. Retries turn a 52% link into a 97% one.

6.5 How strong is CRC-16?

Damage to a 44-byte packet	Trials	Not detected
1 bit flipped	50,000	0
2 bits flipped	50,000	0
3 bits flipped	50,000	0
A burst of up to 16 bits	50,000	0
Heavy damage (8–20 random bits)	200,000	4

CRC-16 is guaranteed to catch small errors and bursts up to 16 bits. With heavy random damage, about 1 in 65,536 slips through by chance (here 4 in 200,000 ≈ 1 in 50,000, as expected). That's why this design has **two** CRCs (frame and packet): a damaged message would have to fool both.

7. Testing and code quality

Test	What it proves	Result
C unit tests (27)	CRC check value and bit-flip detection; big-endian round trips and overflow protection; bit stuffing with worst-case data; three back-to-back frames; corrupted frame dropped while the others survive; packet build/parse, time tags, 14-bit sequence limit, CRC and length errors; TLE checksums.	27 / 27
Python cross-tests (7)	Python and C produce the same bytes; constants come from icd.h; end-to-end NO_OP and wheel command through the real C flight software.	7 / 7
Mission simulation	Behaviour of the whole system over 6.4 h, compared with the simulator's hidden "truth".	All outcomes as designed

Bugs found while building (and what they taught me)

- `memcpy(dst, NULL, 0)` crashed the tests: copying zero bytes from a NULL pointer is undefined behaviour in C. Fixed by returning early for zero length.
- **Time round-trip off by 1 ms**: converting milliseconds to 1/65536 s and back truncated. Fixed by rounding.
- **A big dump would overflow the transmit queue**: fixed by having the transmitter pull stored packets only when it has room (flow control).
- **Time-tagged results were lost out of contact**: execution ACKs and events are now also kept in on-board storage.
- The compiler targeted the wrong CPU at first ("illegal instruction"): always check what architecture you're building for.

8. What I would add next

1. **Run it on real hardware**: port `obc.c` to a RISC-V or ARM board with FreeRTOS (tasks for receive, scheduler, telemetry), using a UART instead of a pipe.
2. **CCSDS COP-1 / FARM-1**: the standard frame-level retransmission protocol, instead of my packet-level retries.
3. **Forward error correction** (Reed–Solomon or convolutional coding) so most bit errors are fixed, not just detected.
4. **Command authentication** (a message authentication code) so nobody else can command the satellite; the duplicate check alone is not security.
5. **ECSS PUS services**: map my ACKs to PUS service 1 and housekeeping to service 3.
6. **A real orbit propagator** (SGP4) using the uploaded TLE, to predict passes and eclipses instead of using a fixed pattern.

9. How to talk about this project

30-second version

"I built a complete CubeSat command and telemetry system from scratch: C flight software with HDLC bit-stuffed framing, CCSDS packets, a seven-gate command validation pipeline, two-stage acknowledgements, a time-tagged scheduler, battery-driven safe mode and stored telemetry; plus an independent Python ground station and a noisy-link simulator. I ran a 6.4-hour simulated mission and a bit-error sweep: measured frame survival matched (1−BER)^N theory, and automatic retries kept command delivery at 97% at a bit error rate of one in a thousand. 34 automated tests, all passing."

Questions you can now answer from experience

- Why does a damaged command get no ACK at all? (Gate 1: you can't trust any field of it.)
- How do you make retries safe? (Same sequence count + duplicate check on board.)
- Why re-check the mode when a time-tagged command runs? (Section 6.1: it was aborted for exactly that reason.)
- Why is stored telemetry essential? (87% of the mission was only seen through dumps.)
- What does CRC-16 guarantee, and what doesn't it? (Section 6.5.)
- How did you keep the ground and the satellite consistent? (icd.h is read by both.)